Utilizando **Java 17 ou superior, Spring Boot e Maven**, desenvolver uma solução baseada em microsserviços responsável pelo gerenciamento de clientes.

A solução deverá permitir cadastrar, alterar, excluir e consultar clientes através de APIs REST.

Além das operações de clientes, a solução deverá:

- realizar integração HTTP com um serviço externo de Score;
- possuir autenticação e autorização baseada em OAuth 2.0/JWT;
- publicar eventos relacionados aos clientes;
- consumir mensagens de forma assíncrona através do RabbitMQ;
- ser executada utilizando containers.

Dados do Cliente

O cliente deverá possuir, no mínimo:

```
{
 "id": 1,
 "name": "João da Silva",
 "cpf": "12345678901",
 "email": "joao@email.com",
 "status": "ACTIVE"
}
```

Caso considere necessário, outros atributos poderão ser adicionados.

APIs de Clientes

Implementar, no mínimo:

POST /customers

PUT /customers/{id}

DELETE /customers/{id}

GET /customers/{id}

GET /customers

Também deverá existir uma consulta por nome:

GET /customers/search?name=joao

E uma consulta por status:

GET /customers?status=ACTIVE

Consulta de Score

Implementar:

GET /customers/{id}/score

Para responder essa requisição, a aplicação deverá consultar um serviço externo utilizando o CPF do cliente.

O serviço externo possuirá uma operação equivalente a:

GET /scores/{cpf}

Exemplo:

```
{
  "cpf": "12345678901",
  "score": 750,
  "classification": "LOW_RISK"
}
```

Não é necessário desenvolver uma aplicação completa para representar esse serviço.

O candidato poderá utilizar WireMock, MockServer, outro microsserviço ou uma solução equivalente.

A solução deverá considerar os diferentes cenários que podem ocorrer durante a comunicação com esse componente externo.

Segurança

As APIs deverão estar protegidas utilizando:

- Spring Security;
- OAuth 2.0;
- JWT.

A aplicação deverá atuar como um Resource Server.

Deverão existir pelo menos dois perfis:

USER

ADMIN

O perfil USER poderá realizar operações de consulta.

Operações de criação, alteração e exclusão deverão possuir autorização apropriada.

Não é necessário desenvolver uma aplicação Web para autenticação.

O candidato poderá utilizar uma solução de identidade de sua preferência ou uma estratégia equivalente que permita obter um Access Token e consumir as APIs utilizando Postman, Insomnia ou ferramenta similar.

Processamento Assíncrono

Quando um novo cliente for cadastrado com sucesso, a aplicação deverá publicar uma mensagem no RabbitMQ representando o evento:

CUSTOMER_CREATED

A mensagem deverá possuir informações suficientes para identificar o evento e o cliente relacionado.

Exemplo conceitual:


```
{
 "eventId": "7ba85b17-7d33-4d17-9101-569a362239f2",
 "eventType": "CUSTOMER_CREATED",
 "customerId": 123,
 "createdAt": "2026-08-11T15:30:00Z"
}
```


A aplicação também deverá possuir um consumidor capaz de receber uma mensagem solicitando alteração do status de um cliente.

Exemplo:


```
{
 "eventId": "cbca5352-22ad-48f2-aaf2-704735bc7737",
 "eventType": "CUSTOMER_STATUS_CHANGE",
 "customerId": 123,
 "status": "INACTIVE"
}
```

Uma mesma mensagem poderá eventualmente ser entregue mais de uma vez.

A aplicação deverá manter seus dados consistentes mesmo nesse cenário.

Requisitos

Utilizar:

- Java 17 ou superior;
- Spring Boot;
- Maven;
- banco de dados relacional;
- Spring Data JPA;
- Spring Security;
- OAuth 2.0 / JWT;
- integração HTTP com serviço externo;
- RabbitMQ;
- publicação e consumo de mensagens;
- validação dos dados recebidos;
- tratamento de erros;
- configurações externalizadas;
- JUnit;
- Docker.

A escolha do banco de dados fica a critério do candidato.

Comportamento da Aplicação

Devem ser consideradas situações como:

- dados inválidos;
- CPF já cadastrado;
- cliente inexistente;
- usuário sem autenticação;
- usuário sem autorização;
- indisponibilidade do serviço externo;

- demora excessiva na resposta do serviço externo;
- indisponibilidade temporária de componentes da solução;
- falha no processamento de uma mensagem;
- recebimento da mesma mensagem mais de uma vez.

As respostas e o comportamento da aplicação deverão ser adequados a cada situação.

Testes

Implementar testes automatizados contemplando as principais funcionalidades, regras de negócio e integrações da solução.

Não é necessário atingir um percentual específico de cobertura.

Containers

A solução deverá poder ser executada localmente através de containers.

As dependências necessárias para o funcionamento da aplicação deverão possuir uma forma documentada de inicialização.

É esperado que seja possível iniciar o ambiente de maneira simples, por exemplo:

docker compose up

ou através de solução equivalente.

Entrega

Ao finalizar, disponibilizar o projeto em um repositório Git acessível aos entrevistadores.

O projeto deverá possuir um README.md contendo, no mínimo:

- arquitetura da solução;
- requisitos para execução;
- como iniciar a aplicação;
- como iniciar as dependências;
- como obter ou utilizar o Access Token;
- como executar os testes;
- endpoints disponíveis;
- configurações necessárias;

- exemplos de utilização das APIs;
- funcionamento da integração com RabbitMQ.